EXPERIMENT / PRACTICAL RECORD

AGILE PROJECT MANAGEMENT AND CI/CD PIPELINE

Using Jira, GitHub, Jenkins and Docker

Name	Khan Ashhad
Roll No.	48
Experiment	06:Agile project setup, version control, CI automation and deployment workflow
Repository	student-management
Primary Jira Sprint	SM Sprint 1

1. Aim

To design and document an Agile software-development workflow in Jira, manage source code using Git and GitHub, automate build execution using Jenkins, and establish the foundation for containerized deployment using Docker.

2. Objectives

- Create an Agile project and manage user stories through a Scrum sprint in Jira.
- Develop a small Student Management System login interface as the working application.
- Initialize a Git repository, create meaningful commits, and push the project to GitHub using the main branch.
- Configure Jenkins to retrieve the GitHub repository and execute a repeatable Windows build step.
- Configure a GitHub webhook so that a push can trigger Jenkins automatically.
- Maintain evidence of the development workflow using screenshots at significant milestones.
- Document the Docker deployment procedure specified by the experiment instructions.

3. Theory

3.1 Agile and Scrum

Agile is an iterative software-development approach in which work is divided into small, manageable units and delivered incrementally. Scrum is a commonly used Agile framework. A Scrum project organizes work into a backlog, user stories and time-boxed sprints. A Scrum board provides visual tracking through states such as To Do, In Progress and Done. This makes the status of work visible and allows unfinished work to be identified early.

In this experiment, the Student Management System is represented by Jira work items. SM Sprint 1 contains four work items. Completed work items are moved through the status flow To Do -> In Progress -> Done. Jira can then provide sprint-level reports such as a burndown chart, sprint report and velocity chart after the sprint is completed.

3.2 Git and GitHub

Git is a distributed version-control system. A Git repository stores the history of a project as commits. The staging area allows selected changes to be prepared before a commit. A remote repository on GitHub provides centralized online access to the same project history. In this experiment, the local repository was connected to the GitHub repository student-management and the primary branch was main.

The basic cycle is: modify files -> git add -> git commit -> git push. Commit messages were written to describe the change and, where appropriate, reference the corresponding Jira work item.

3.3 Continuous Integration with Jenkins

Jenkins is an automation server used to execute repeatable software-development tasks. A Jenkins job can retrieve source code from a version-control repository, execute build commands and report the

result. Continuous Integration (CI) means integrating changes frequently and verifying them automatically. In this experiment, Jenkins performs a Windows batch build that checks out the main branch, lists the workspace contents and reports a successful build.

The important pipeline relationship is: GitHub repository -> Jenkins job -> build step -> build result. The experiment further extends this by using a GitHub webhook, so a successful push can trigger Jenkins without manually pressing Build Now.

3.4 GitHub Webhooks

A webhook is an HTTP callback through which one service sends an event notification to another service. A GitHub push event can be sent to a Jenkins endpoint. Because this practical uses Jenkins on localhost, a tunneling service was used in the captured setup to make the local Jenkins endpoint temporarily reachable from GitHub. The tunnel forwards public HTTPS requests to the local Jenkins port.

3.5 Docker

Docker packages an application and its runtime environment into a container image. A Dockerfile describes how the image is assembled. The experiment specifies an Nginx image, copies the project files into the Nginx web directory, builds the image as student-management, and runs it with host port 8080 mapped to container port 80.

4. Software / Requirements

- Jira account and Scrum project
- Git and GitHub account
- Java 21 LTS / JDK 21 for Jenkins
- Jenkins 2.568.2
- ngrok for the captured local-webhook test
- Docker Desktop / Docker CLI
- Windows Command Prompt and a web browser

5. Experimental Workflow

Phase	Tool	Major Action	Evidence
Planning	Jira	Create stories and sprint	Jira board screenshots
Version control	Git/GitHub	Commit and push source	Terminal/GitHub screenshots
CI	Jenkins	Configure job and run builds	Console Output screenshots
Automation	GitHub + Jenkins	Webhook-triggered build	Webhook + automatic build screenshots
Tracking	Jira	Move completed work to Done	Backlog screenshots
Deployment	Docker	Build and run container	Docker/application screenshots

6. Procedure: Version Control Setup

The source project was maintained under C:\Users\15L\student-management. Git was initialized earlier, the main branch was used, and the GitHub repository served as the remote source of truth. The evidence confirms that index.html was pushed successfully to the repository.

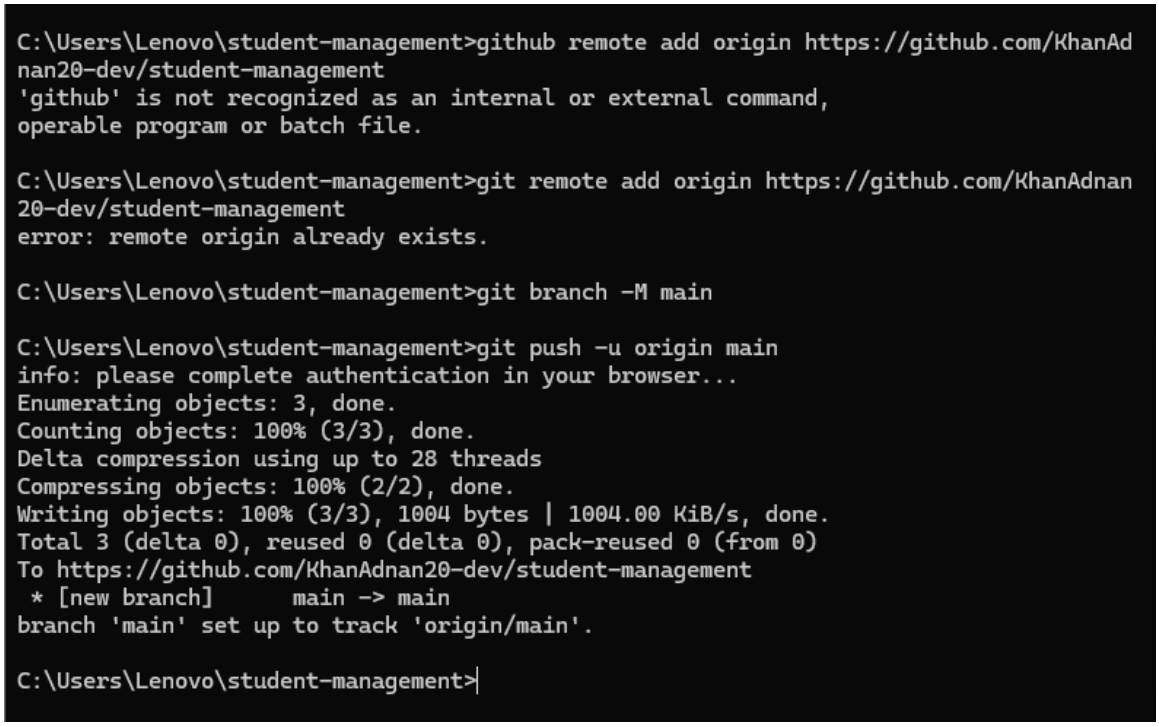


Figure 1. Version-control setup: first project push

The screenshot records the local Git repository and successful push of the initial index.html code to the GitHub main branch.



Figure 2. Jenkins installation completed

The Jenkins 2.568.2 Setup Wizard is shown at the completed-installation stage.

7. Procedure: Jenkins Installation and Initial Configuration

Jenkins was installed on Windows with Java 21. The service was configured so that the Jenkins web interface could run locally. The Jenkins Freestyle job was then connected to the GitHub repository and a Windows batch build step was added. The screenshots document installation, trigger configuration and successful console execution.

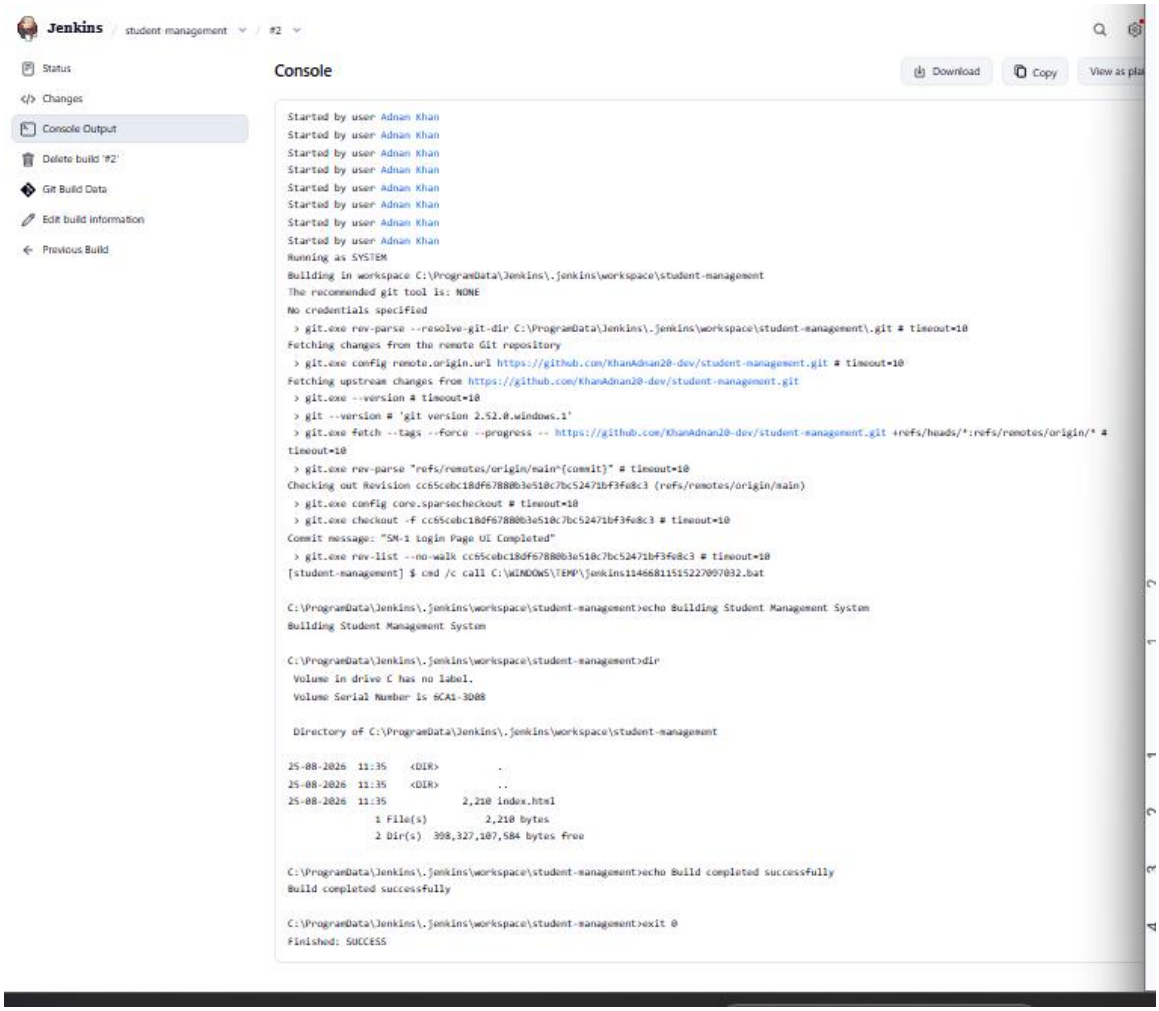


Figure 3. Initial Jenkins build success

The Jenkins Console Output demonstrates a successful build and ends with Finished: SUCCESS.

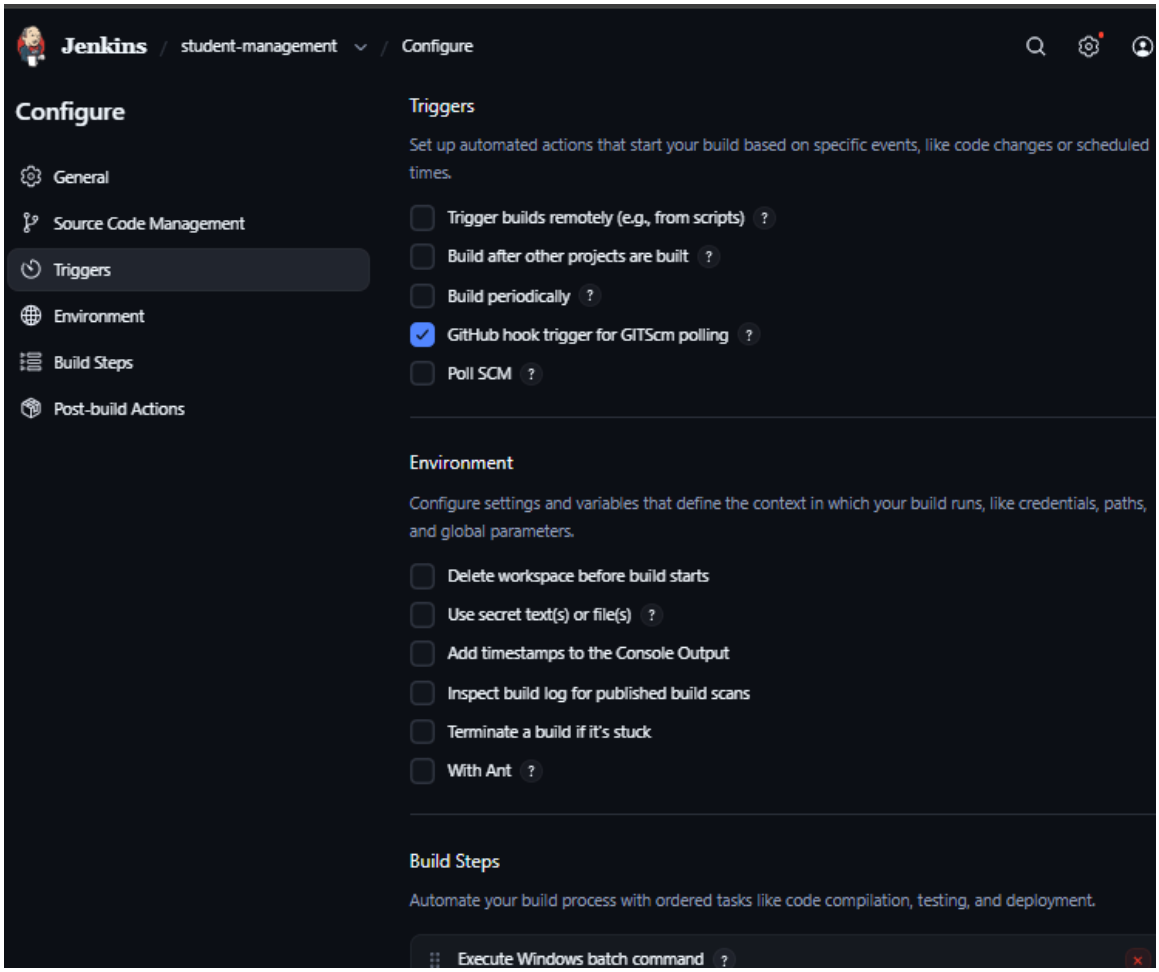


Figure 4. Jenkins automatic-trigger configuration

The Jenkins job configuration shows GitHub hook trigger for GITScm polling enabled.



Figure 5. Jenkins build evidence after configuration

The captured Jenkins console provides additional evidence of repository checkout and batch execution.

8. Procedure: GitHub Webhook and Automatic Build

The Jenkins trigger was configured for GitHub hook events. A local Jenkins server is not directly reachable from GitHub, so the captured workflow used an ngrok Agent Endpoint to forward a public HTTPS URL to the local Jenkins port. The GitHub webhook was then configured for the push event. Subsequent pushes were used to verify automatic Jenkins execution.

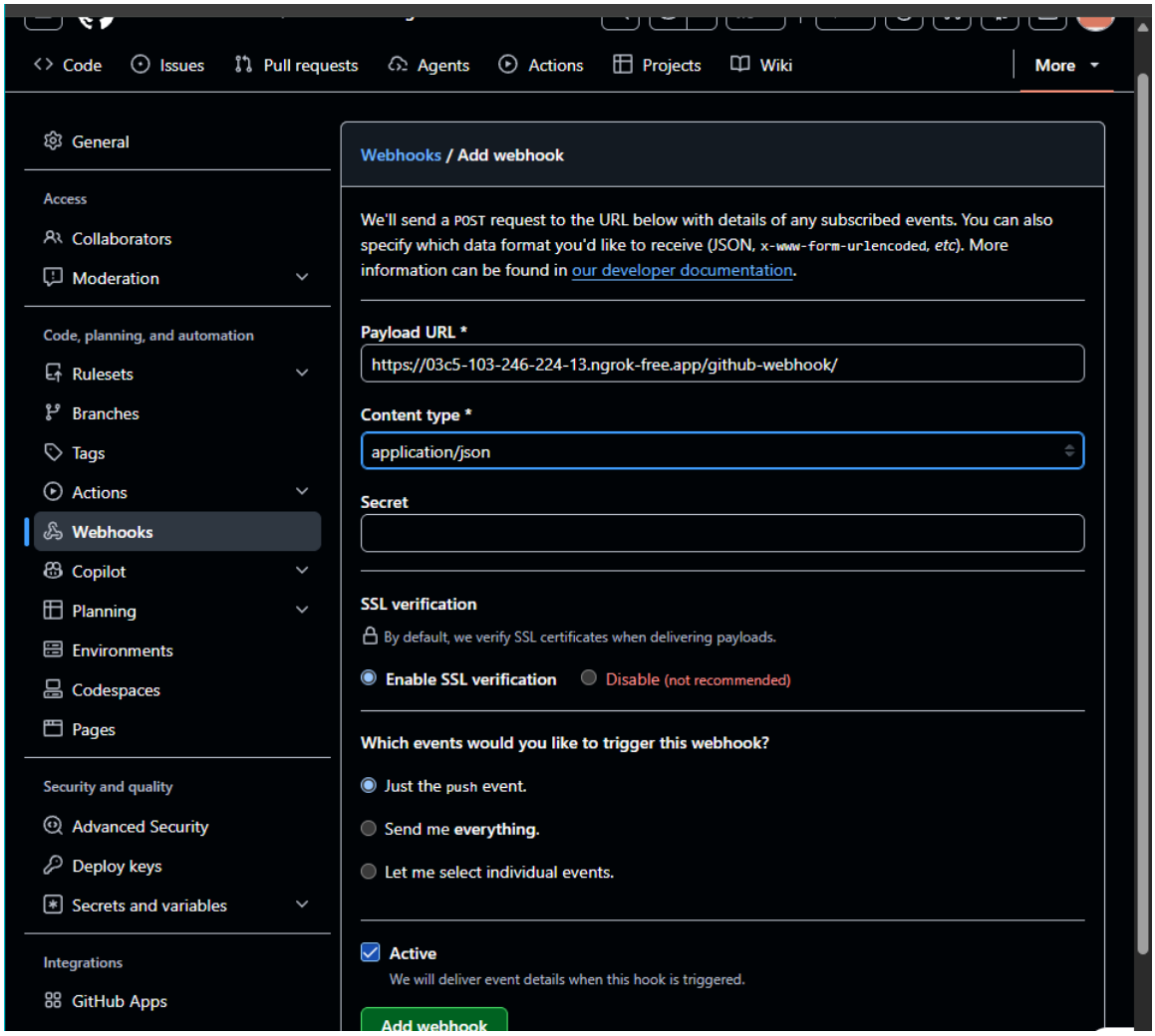


Figure 6. GitHub webhook configuration

The GitHub Webhooks page shows the webhook configured to send push-event information to Jenkins through the public tunnel URL.

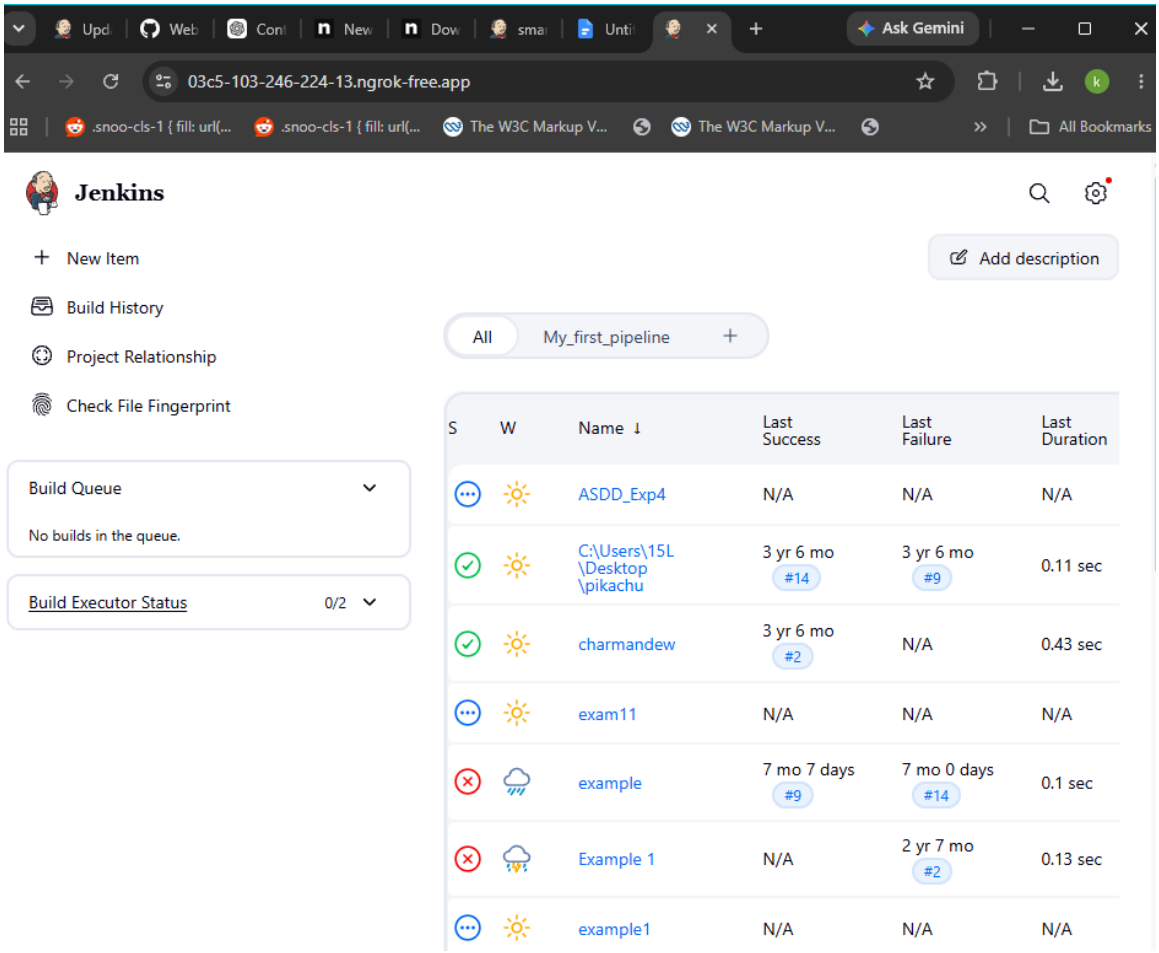


Figure 7. Public access to Jenkins through ngrok

The browser successfully opens the Jenkins dashboard through the ngrok public URL, demonstrating that the local Jenkins instance is reachable externally.

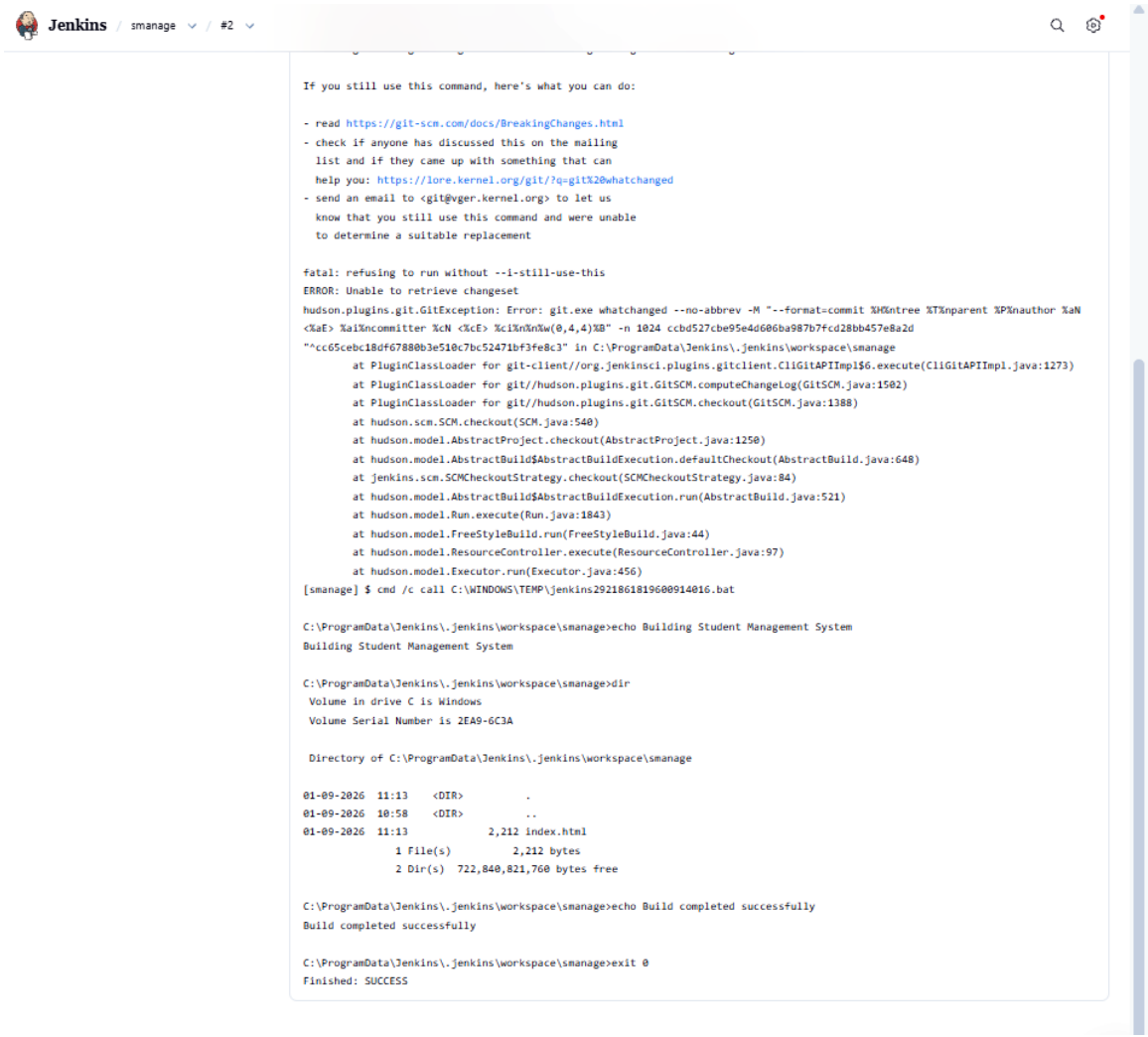


Figure 8. Automatic Jenkins build triggered from GitHub

The Jenkins console shows the repository being retrieved after a GitHub event; this documents the webhook-driven automation path.

9. Procedure: Jira Work Tracking and Database Schema Task

Jira was used as the planning and tracking layer. The evidence shows Sprint 1 containing the Student Management work items. Because PostgreSQL itself was not installed for this captured run, the database story was reframed as a database-schema design task and represented by a database-schema.sql file. This created a tangible deliverable without claiming that a PostgreSQL server was deployed.

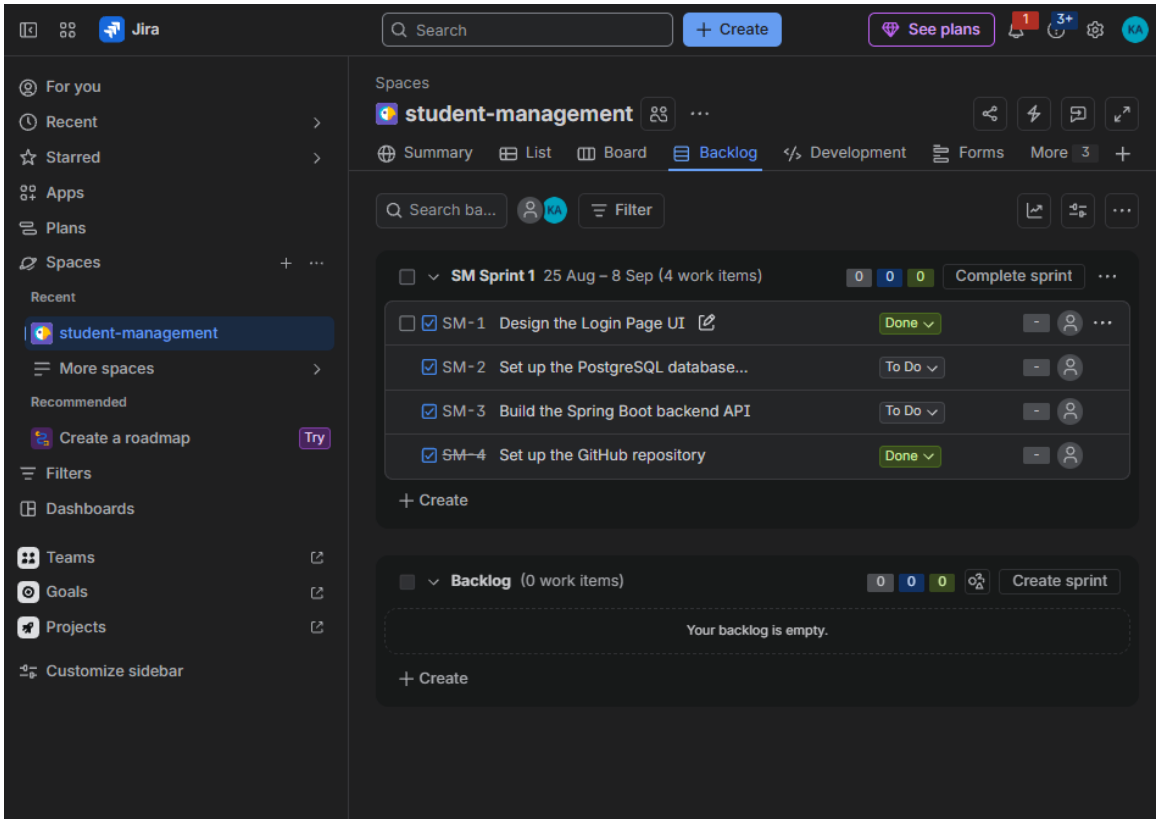


Figure 9. Jira Sprint 1 work-item tracking

The Jira backlog displays SM-1 through SM-4 in SM Sprint 1 and shows completed and pending statuses.

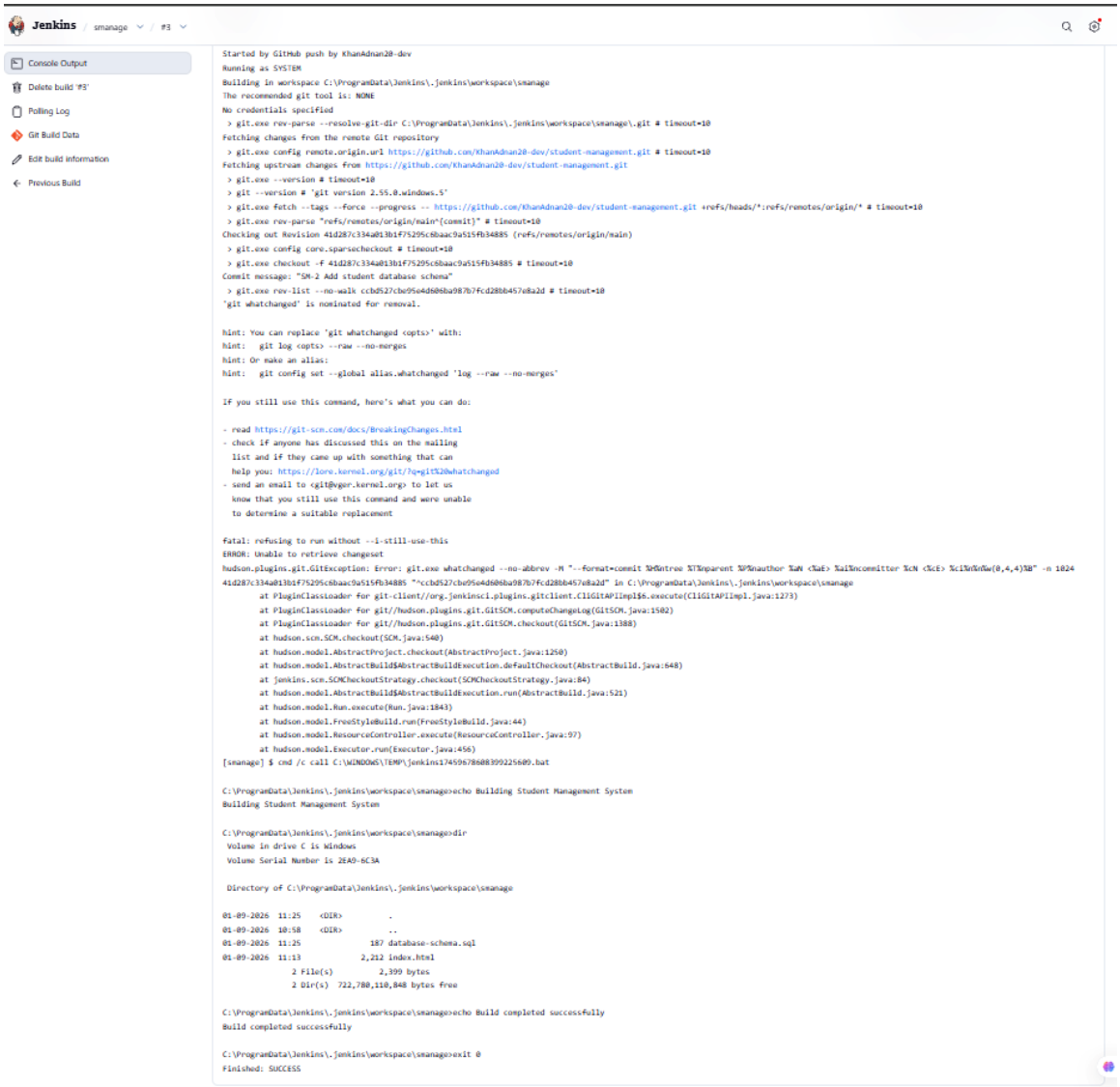


Figure 10. Database schema work integrated into Jenkins

The Jenkins console shows database-schema.sql alongside index.html and finishes with SUCCESS, demonstrating that the additional work entered the CI flow.

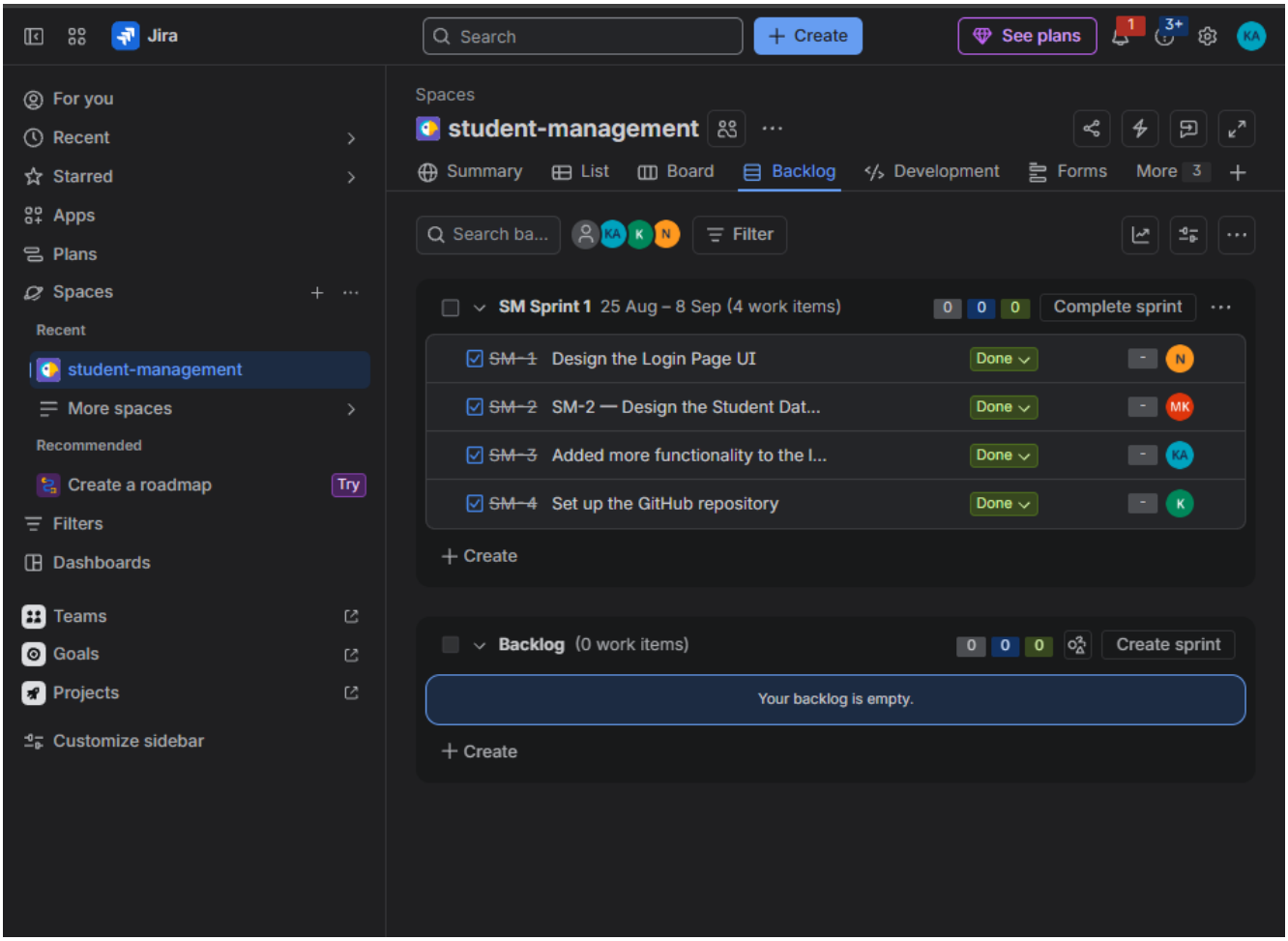


Figure 11. Jira work-item status updated

The Jira board records updated work-item statuses after the repository/automation work was completed.

10. Procedure: Functional Login-Page Enhancement

The login page was enhanced with client-side functionality, including input validation, password visibility control, remember-me behavior and demo authentication feedback. The updated file was committed to GitHub and the webhook automatically triggered a Jenkins build.

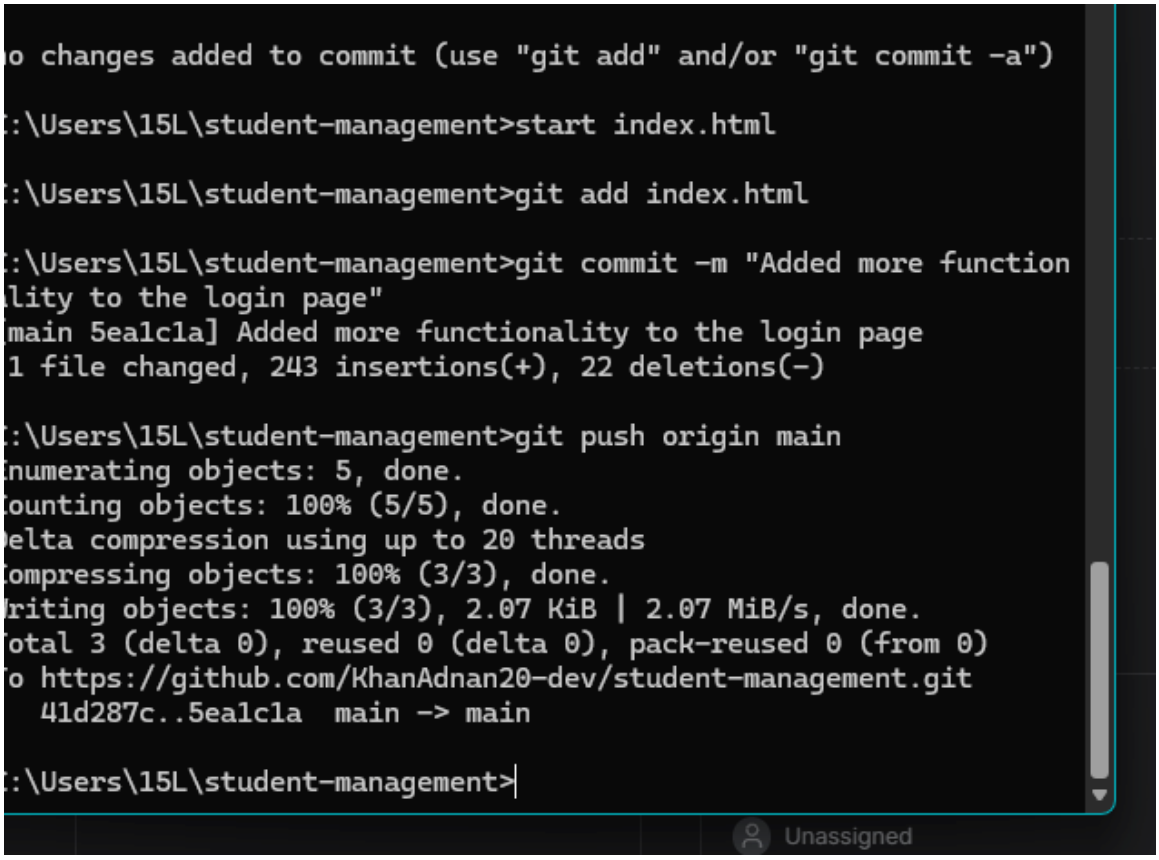


Figure 12. Login-page functionality committed and pushed

The terminal screenshot shows the updated login-page functionality being committed and pushed to the GitHub main branch.

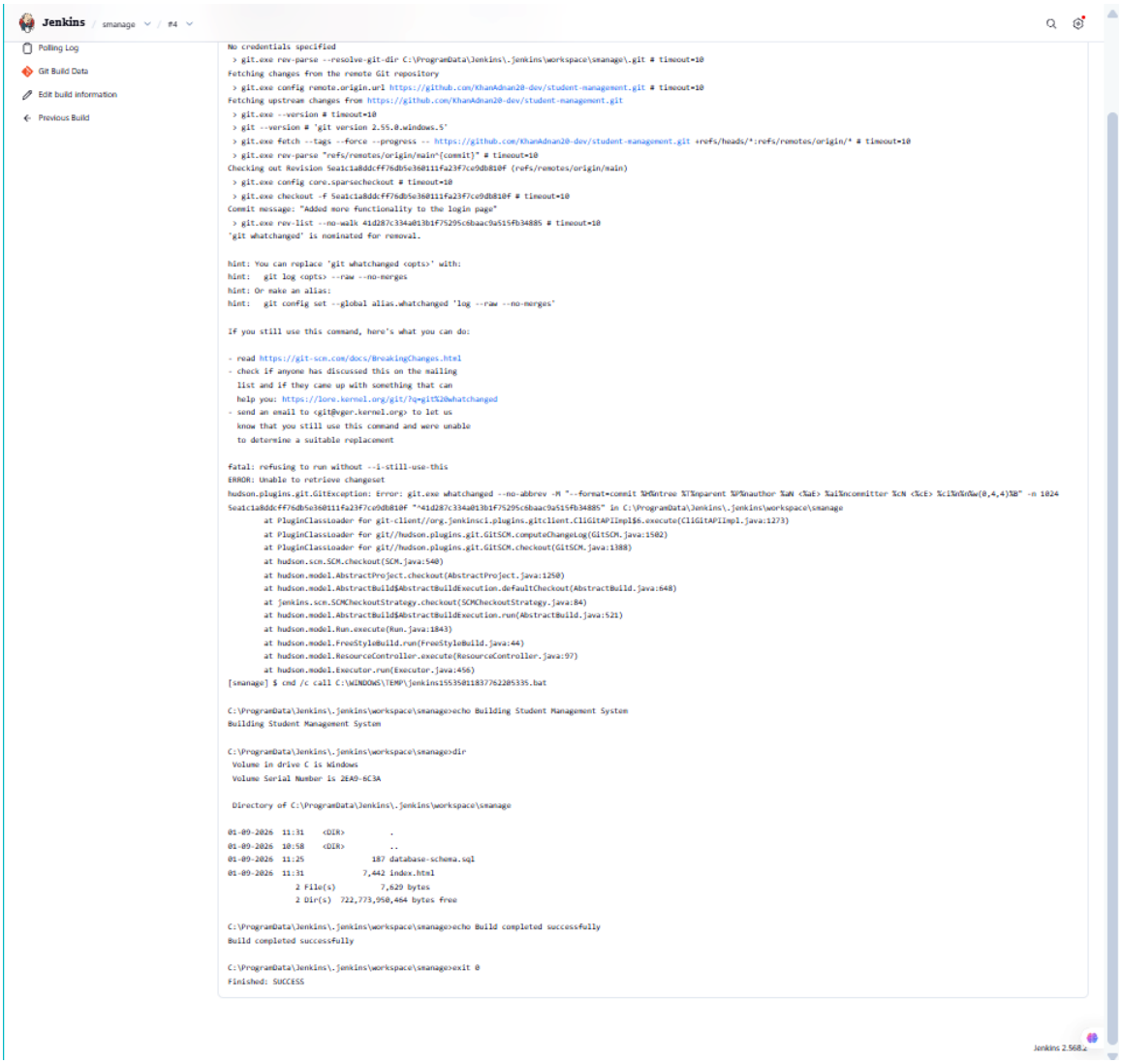


Figure 13. Jenkins automatic build after login-page update

The Jenkins console shows the GitHub-triggered build for the login-page update and a successful build result.

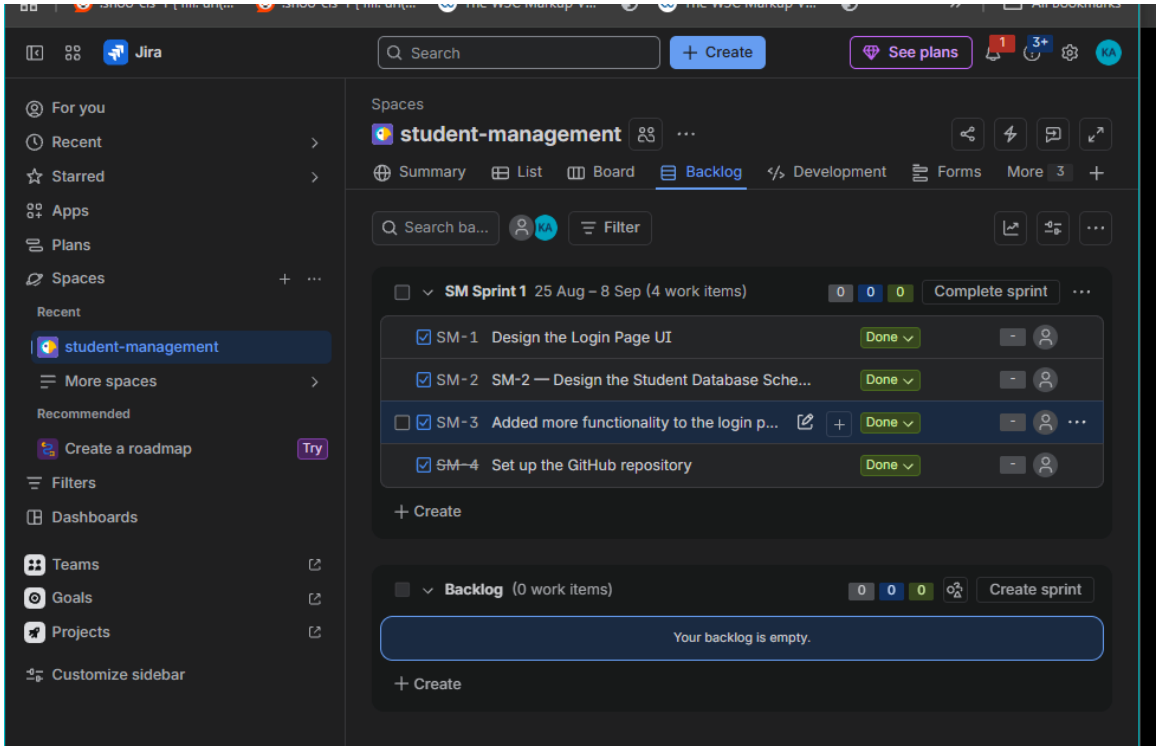


Figure 14. Final Jira status update captured

The final Jira screenshot records the updated sprint board after the latest development work.

11. Important Commands Used

Purpose	Command	Explanation
Check Git	git --version	Verifies the Git installation.
Initialize repository	git init	Creates/initializes the local Git repository.
Stage file	git add index.html	Places the modified file in the staging area.
Commit change	git commit -m "SM-1 Test automatic Jenkins build"	Creates a versioned snapshot with a descriptive message.
Connect remote	git remote add origin https://github.com/KhanAdnan20-dev/student-management.git	Associates the local repository with GitHub.
Rename branch	git branch -M main	Ensures the local primary branch is main.
Push code	git push origin main	Uploads the local commit history to GitHub.
Start Jenkins webhook tunnel	ngrok http 8081	Forwards a public ngrok URL to the local Jenkins port used in the captured setup.
Docker image build	docker build -t student-management .	Builds the container image from the Dockerfile.
Run Docker container	docker run -p 8080:80 student-management	Maps host port 8080 to Nginx port 80 in the container.

12. Docker Deployment Procedure Specified by the Experiment

The provided experiment instructions specify the following deployment procedure: create a Dockerfile using nginx:latest, copy the project into /usr/share/nginx/html, build the image as student-management, run it with port mapping 8080:80, and access the result at http://localhost:8080. The source screenshot set supplied with this report does not contain a Docker build/run screenshot, so this section documents the required procedure without falsely claiming that the Docker execution was captured.

```
FROM nginx:latest
COPY . /usr/share/nginx/html

docker build -t student-management .

docker run -p 8080:80 student-management
```

13. Result

The captured practical work successfully demonstrates an Agile Jira workflow, Git/GitHub version control, Jenkins Continuous Integration, and a working GitHub webhook that automatically triggered Jenkins after a push. The Jenkins console evidence repeatedly shows Finished: SUCCESS. The Jira board evidence documents the progression of the Student Management work items. Docker installation was present on the working system, while the supplied screenshot evidence does not include a Docker build/run execution; therefore Docker deployment is documented as the remaining verification item rather than presented as completed evidence.

14. Conclusion

This experiment demonstrates how project planning, source-code management, continuous integration and Agile work tracking can be connected into one practical development workflow. Jira provides visibility of work, Git and GitHub provide source history and remote storage, Jenkins provides automated verification, and the webhook closes the loop by allowing source changes to trigger automation automatically. Docker provides the final packaging and deployment mechanism specified by the experiment.